ML WORKING REFERENCE

Dataset → Model

From a business question to a deployed model: the lifecycle up top, the branching decision core in the middle, and lookup matrices, a scoring rubric, and checklists below. Colors stay consistent across the flowchart and every table.

EDA / neutral · Supervised · Unsupervised · Reinforcement

01
Frame the question
Business goal + what a "win" looks like

02
EDA
Explore shape, distributions, relationships

03
Data quality
Missing, dupes, leakage, ranges

04
Define the task
Target? Type? → paradigm

05
Preprocess
Split first, clean, impute, encode

06
Feature engineering
Create the signal the model learns from

07
Baseline
Naive floor (mean / mode), then a simple model

08
Stronger models
Ensembles, then compare

09
Evaluate
Right metric, honest validation

10
Explain
SHAP, importances, coefficients

11
Deploy / iterate
Serve, monitor, loop back

The highlighted steps — define the task, feature engineering, baseline, and stronger models — are where the flowchart and matrices below go deep. The rest is covered by the rubric and checklists at the bottom.

PLAYBOOK

The 11 steps, worked through

Each step with what it actually means and one running example: **predicting the fare of a NYC taxi trip at the moment of pickup**. Steps 01 and 09 get extra room — framing and "what counts as success" are where most projects quietly go wrong.

01 FRAME

Frame the question

Turn a vague wish ("predict fares") into a precise, decidable question. Before touching a model, pin down:

- → **The decision it informs** — who acts on the output, and what changes because of it? (Show riders an upfront price.)
- → **Unit of prediction** — one row = one what? (One taxi trip.)
- → **The exact target** — what column, in what units? (Final fare in dollars.)
- → **What's known at prediction time** — only features available *before* the ride. Using anything from during/after the trip is leakage.
- → **Cost of being wrong** — and which direction hurts more. (Over-quoting loses riders; under-quoting loses money.)

Then define success on two levels — an *ML metric* (how you score the model) and a *business metric / threshold* (what makes it worth shipping), plus the status-quo bar you must beat.

EXAMPLE

Predict a trip's fare at pickup, per trip, from pickup time + zones + passenger count, so the app shows a reliable upfront price. `SUCCESS:` within $2 of actual on 80% of trips (ML: MAE / RMSE) *and* beats today's flat-rate estimate; inference under 100 ms.

02 EDA

EDA

Understand the data's shape and surprises before modeling — distributions, relationships, and anything weird.

EXAMPLE

Plot the fare distribution (catch negative fares, $0 rides, 200-mile outliers), check fare-vs-distance correlation, look at trips-by-hour patterns.

03 QUALITY

Data quality

Catch what quietly wrecks a model: missing values, duplicates, impossible values, and leakage.

EXAMPLE

Drop trips with negative fare or zero distance; verify `total_amount` isn't just fare + tip — tip isn't known at pickup, so including it would leak the answer.

04
TASK

Define the task

Run the flowchart: is there a target? what type? → paradigm + task. This usually re-confirms step 01. One ambiguity to resolve deliberately: a **numeric target with only a few distinct values** (a 1–5 rating, a 0–10 score) isn't automatically regression. It can be modeled as regression (predict the value), classification (predict the class), or **ordinal** (respect the order) — that's your call to make, not the data's.

EXAMPLE

Fare is a continuous number we have labeled → **supervised regression**. A wine-quality score of 3–8, by contrast, is the ambiguous case: often best treated as ordinal, reporting accuracy-within-1 alongside RMSE.

05
PREP

Preprocess

Make the data model-ready *without* leaking. Split first, then clean, impute missing values, and encode — fitting every transform on the training fold only.

EXAMPLE

Hold out a later time window for testing; impute missing passenger counts; encode pickup zone using one-hot, frequency, or target encoding — fit on train only.

06
FEATURES

Feature engineering

The highest-leverage step for tabular ML: turn raw columns into the signal the model actually learns from. A strong model on weak features loses to a simple model on good ones. Work *by data type* — numeric, categorical, datetime, text — then add cross-feature interactions, aggregates, and missingness indicators where they make domain sense. Keep it iterative: engineer, train, read importances, keep what helps. (See the feature-engineering matrix below for techniques by type.)

EXAMPLE

From a pickup timestamp: hour-of-day, day-of-week, is_weekend, and cyclical sin/cos of the hour. From pickup and destination zones, if both are known at quote time: route distance and mean-fare-by-route encoding fit inside CV. Plus an is-airport-trip flag from the zones. (A "speed" feature would need trip duration — unknown at pickup — so it stays out.)

07
BASELINE

Baseline

Set the floors to beat: a *naive* predictor first (`DummyRegressor` = predict the mean), then a *simple* model.

EXAMPLE

"Always predict the average fare" gives some RMSE; a linear regression on distance + time gives a better one. That number is now the bar.

08
MODELS

Stronger models

Bring in ensembles and compare against the baseline. Diagnose first: if variance is high (overfitting), try bagging / Random Forest or more regularization; if bias is high (underfitting), try a more expressive model like boosting — which itself needs care not to overfit.

EXAMPLE

Try Random Forest, then LightGBM; compare cross-validated RMSE against the linear baseline.

09
EVALUATE

Evaluate — what "success" really means

Score on the **right metric** with **honest validation**, and check the business threshold from step 01, not just the loss function. Five checks:

- → **ML metric** — regression: RMSE / MAE / R^2. Classification: PR-AUC / F1 / recall (not accuracy if imbalanced).
- → **Business metric** — the number a stakeholder cares about. Here: "% of trips quoted within $2."
- → **Calibration** — when the *probability* matters more than the label (risk score, fraud, churn, readmission), check a reliability curve and calibrate with Platt or isotonic.
- → **Subgroup performance** — in regulated or high-stakes settings, compare metrics across subgroups (age, sex, geography, payer type, etc.) where legally and ethically permitted, not just the overall average.
- → **Validation that doesn't lie** — proper cross-validation; for anything time-dependent, split by time (walk-forward), never shuffle.

EXAMPLE

Report MAE in dollars *and* "78% of trips within $2," validated on a later time window than training — because traffic and pricing patterns drift.

10
EXPLAIN

Explain

Make the model's reasoning legible with SHAP, feature importances, or coefficients. Builds trust — and catches leakage.

EXAMPLE

SHAP shows distance and hour dominate (sensible). If a dropoff-only feature dominates a *pickup-time* prediction, suspect leakage.

11
DEPLOY

Deploy / iterate

Serve it, monitor for drift, loop back. A model stuck in a notebook helps no one.

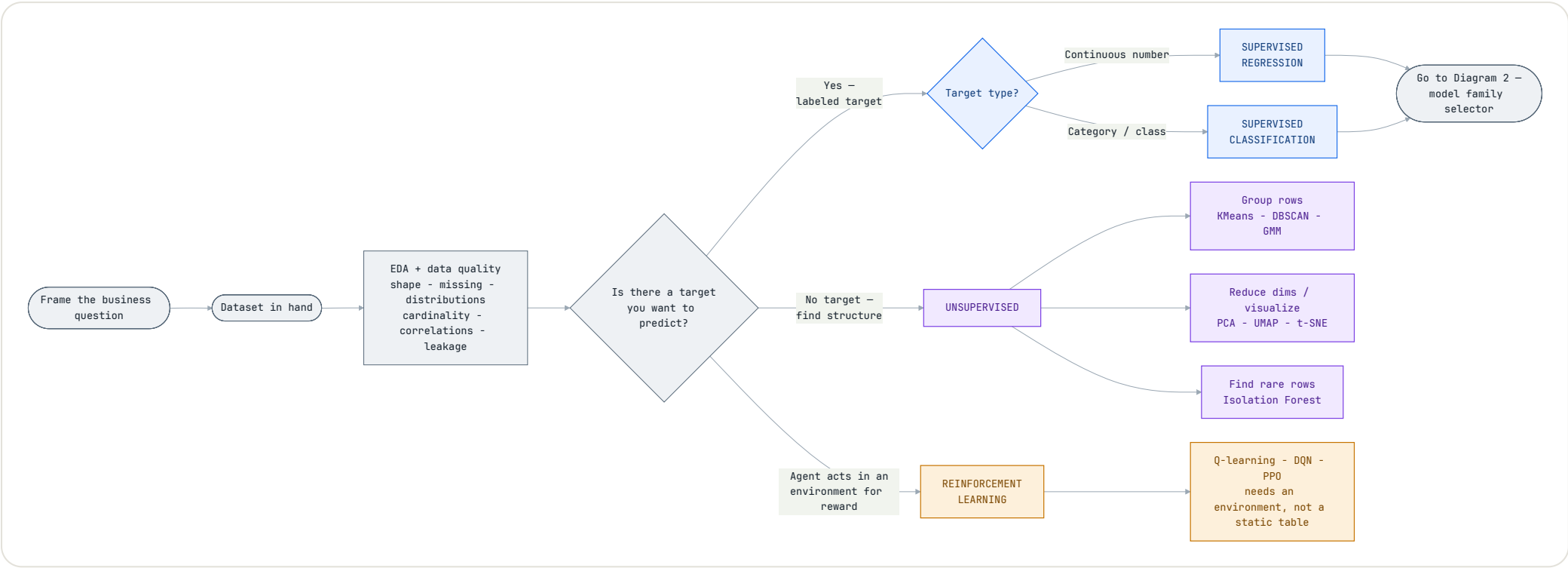
EXAMPLE

Wrap it in a FastAPI endpoint, log predicted vs actual fares, and retrain when error creeps up (track runs in MLflow).

FLOW · A

Diagram 1 — Paradigm selector

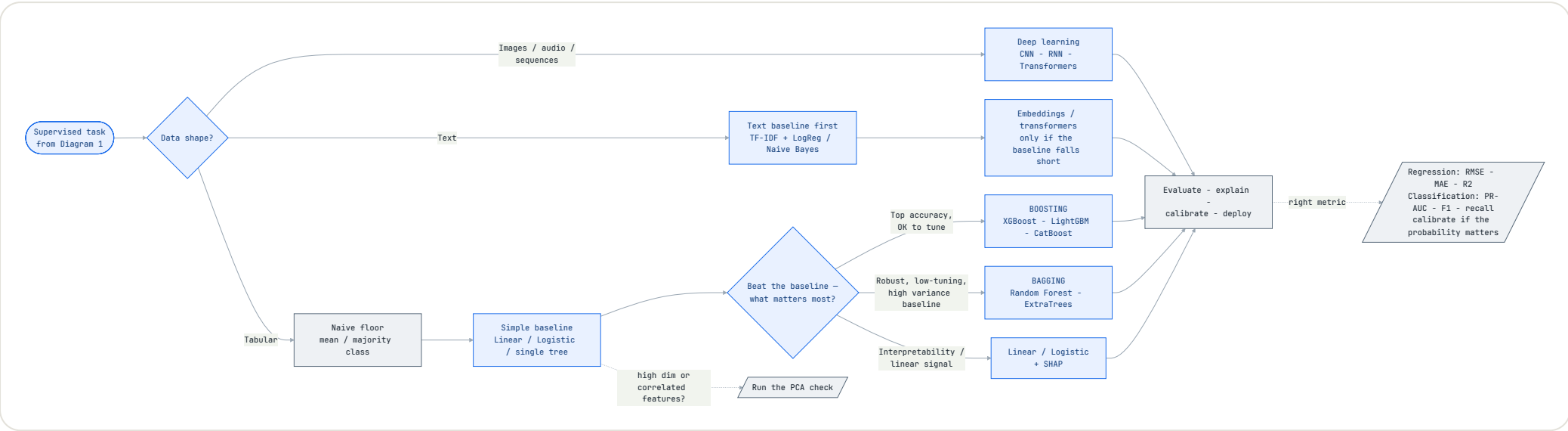
The first half of the decision: from a dataset to *which kind of problem* you have. The first fork — does a target exist? — resolves almost everything downstream.



FLOW · B

Diagram 2 — Model family selector (supervised)

Picking up where Diagram 1 lands you on a supervised task: choose the model family, with the baseline ladder built in before any ensemble.

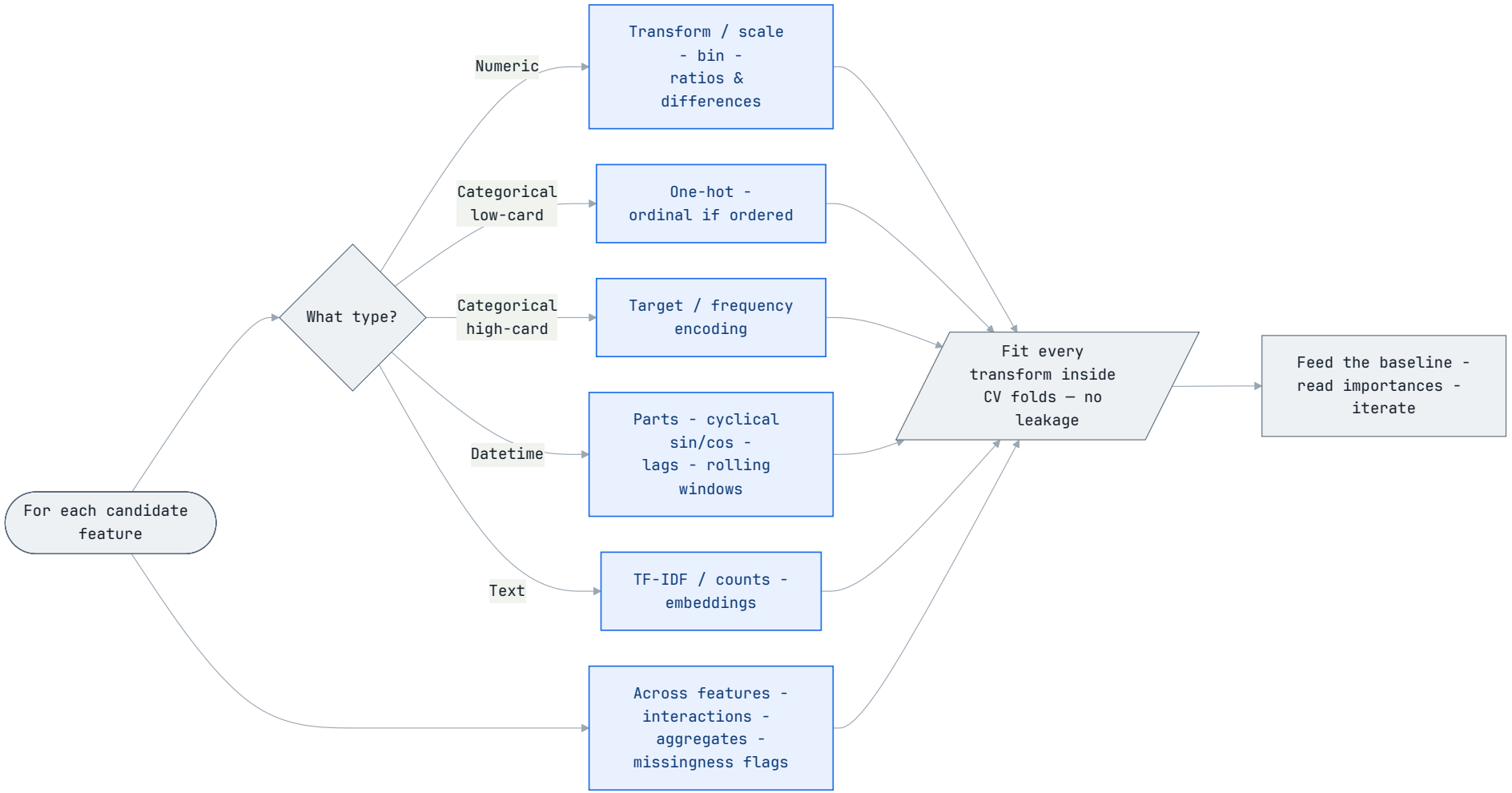


The baseline ladder. Before any ensemble, clear two floors: a *naive* baseline (predict the mean for regression, the majority class for classification — sklearn's `DummyRegressor` / `DummyClassifier`), then a *simple* model (linear / logistic). If a tuned XGBoost can't beat the naive floor by a meaningful margin, the problem is the framing or the data, not the model.

FLOW · C

Diagram 3 — Feature engineering by data type

Step 06 in detail: for each raw column, the right techniques depend on its type — and every transform that *learns* from the data belongs inside the CV folds to stay leakage-free.



MATRIX 01

Which learning paradigm?

Match the signal in your data to the paradigm. The presence and type of a target column does the heavy lifting.

WHAT YOU SEE IN THE DATA	PARADIGM	TASK	TYPICAL MODELS
Target column of **continuous numbers** (fare, duration, price)	Supervised	Regression	`LinearReg · RF · XGBoost`
Target column of **categories / labels** (fraud y/n, tier)	Supervised	Classification	`LogisticReg · NaiveBayes · RF · XGBoost`
No target; want natural groupings	Unsupervised	Clustering	`KMeans · DBSCAN · GMM`
No target; too many features / want a 2D view	Unsupervised	Dim. reduction	`PCA · UMAP · t-SNE`
No target; want the odd / rare rows	Unsupervised	Anomaly detection	`IsolationForest · OC-SVM`
Agent taking actions in an environment for a **reward** over time	Reinforcement	Sequential control	`Q-learning · DQN · PPO`

RL is the odd one out. It doesn't start from a static dataset — it needs an interactive environment and a reward signal. A file of rows is almost always supervised or unsupervised, not RL, unless it carries action-and-reward history.

MATRIX 02

Given your situation, start here → level up

The prescriptive shortcut: find your row, ship the baseline, then earn the stronger model only if validation says it's worth the complexity.

YOUR SITUATION	START HERE (BASELINE)	LEVEL UP
Small tabular, classification	`Logistic Reg · Decision Tree`	`Random Forest`
Medium / large tabular	`Random Forest`	`XGBoost · LightGBM · CatBoost`
Lots of categorical features	`CatBoost (native cats)`	`Tuned CatBoost / LightGBM`
Need interpretability	`Linear / Logistic · single tree`	`Tree model + SHAP`
Accuracy is the priority	`Random Forest`	`Tuned gradient boosting`
Imbalanced classes	`LogReg + class weights`	`XGB/LightGBM + scale_pos_weight`
Text	`TF-IDF + Logistic Reg / Naive Bayes`	`Embeddings / transformer + clf`
Images	`Pretrained CNN (transfer)`	`Fine-tuned CNN / ViT`
Time series / forecasting	`Naive · ARIMA · Prophet`	`Lag features + boosting; deep if large`

YOUR SITUATION	START HERE (BASELINE)	LEVEL UP
Very small dataset	`Simple model + cross-validation`	`Light, well-regularized boosting (only if CV says so)`
High-cardinality categoricals (zip, SKU, ID)	`CatBoost / leakage-safe target encoding`	`Tuned CatBoost / LightGBM (encode inside CV folds)`
Need calibrated probabilities	`LogReg (often well-calibrated)`	`RF / boosting + Platt or isotonic calibration`
Healthcare / regulated use case	`Interpretable baseline + calibration`	`Monitored model + subgroup fairness (age, sex, geography, payer type — where permitted) + documentation`
Time-dependent tabular (drifts over time)	`Simple model, time-based split`	`Boosting; only features known at prediction time`

> **Two traps in this table.** For **imbalanced** problems, never judge on accuracy — use PR-AUC, F1, or recall at a fixed precision. For anything **time-dependent**, validate in time order (walk-forward) — shuffling leaks the future into training — and use only features whose values are actually known at prediction time.

MATRIX 03

Why each family behaves the way it does

Your three terms aren't siblings: **Random Forest is itself a bagging method**. The real choice is bagging vs boosting vs simpler/other models.

FAMILY	HOW IT WORKS	BIAS / VARIANCE	REACH FOR IT WHEN	EXAMPLES
Single tree	One decision tree splits the data	Low bias, **high variance**	Fast, fully interpretable baseline	`DecisionTree`
Bagging `incl. Random Forest`	Many trees in **parallel** on bootstrap samples; vote / average	Cuts **variance**	Robust default, noisy data, minimal tuning	`RandomForest · ExtraTrees`
Boosting	Trees built **sequentially**, each fixing the last one's errors	Cuts **bias & variance**	Top accuracy on tabular; willing to tune; cleanish data	`XGBoost · LightGBM · CatBoost`
Linear / Logistic	Weighted sum of features	**High bias**, low variance	Roughly linear signal; interpretability; sparse high-dim	`LinearReg · LogisticReg`
Naive Bayes	Probabilistic; assumes features are independent given the class	**High bias**, low variance	Fast text-classification baseline; high-dim sparse features; very little data	`MultinomialNB · GaussianNB`
Distance-based	Predicts from nearby / separating points	Varies	Small–medium data, smooth boundaries; **scale first**	`kNN · SVM`
Neural nets	Layered non-linear transforms	Very high capacity	Images, text, audio, sequences, or very large data	`CNN · RNN · Transformer`

> **Rule of thumb for tabular.** Random Forest as the honest baseline → then a boosting model to chase accuracy. As a starting direction: **high variance (overfitting) → bagging / Random Forest or more regularization**; **high bias (underfitting) → a more expressive model such as boosting**. Treat that as a hint, not a law — boosting can overfit if tuned too aggressively, and a Random Forest can underfit if its trees are over-constrained. Either way, cross-validate 2–3 candidates and let the score decide.

MATRIX 04

If it's unsupervised: pick by goal and shape

No target means the goal itself is the decision. Scale features first for anything distance-based.

GOAL	PICK	WHEN IT FITS
Group records	`KMeans`	Roughly round clusters, K known or estimable
Group records	`DBSCAN / HDBSCAN`	Irregular shapes, noise, unknown K
Group records (soft)	`Gaussian Mixture`	Want probability of membership, not hard labels
Reduce dims (linear)	`PCA`	Compression, denoise, speed up downstream model
Reduce dims (visualize)	`UMAP / t-SNE`	Nonlinear structure, 2D/3D plots only — not as model input
Find odd rows	`Isolation Forest`	Tabular default
Find odd rows	`One-Class SVM`	Smaller datasets
Find odd rows	`Autoencoder`	Large, complex, high-dimensional data

MATRIX 05

Do you actually need PCA?

Decided by **feature count, correlation, and model type** — not row count. A million rows with 8 clean features needs no PCA; 300 rows with 4,000 features probably does.

SITUATION	PCA?	WHY
Many features relative to rows (high `p`)	CONSIDER	Compresses signal, fights the curse of dimensionality
Lots of **correlated / redundant** numeric features	CONSIDER	PCA components are uncorrelated by construction
Model is **distance- or linear-based** (kNN, SVM, KMeans, linreg)	OFTEN HELPS	These degrade as dimensionality grows
You need a **2D / 3D plot**	YES (viz)	Project to 2–3 components to see structure
Model is a **tree ensemble** (RF, XGBoost)	USUALLY SKIP	Trees handle correlation natively; PCA kills feature importances
Few, already-meaningful features	SKIP	Nothing to gain; you lose clarity
Interpretability required	BE CAREFUL	Components are opaque linear blends

SITUATION	PCA?	WHY
Sparse high-dim text (TF-IDF)	USE SVD	TruncatedSVD instead — PCA assumes dense, centered data

For tree-based work specifically: keep PCA out of the main pipeline. It pays off as a *side experiment* — compressing a wide one-hot block, or a 2-component scatter to eyeball clusters — not as default preprocessing.

MATRIX 06

Feature engineering — techniques by data type

The reference for step 06, and the companion to Diagram 3. Match each column's type to its techniques, and mind the trap each one hides — most are leakage if fit before the split.

FEATURE TYPE	COMMON TECHNIQUES	WATCH OUT FOR
Numeric	Log / Box-Cox transforms, scaling, binning, ratios & differences, polynomial terms	Trees don't need scaling; fit scalers on train only
Categorical — low cardinality	One-hot; ordinal only if genuinely ordered	One-hot explodes width with many levels; ordinal implies a false order
Categorical — high cardinality (zip, SKU, ID)	Target / mean encoding, frequency / count encoding, hashing; or native (CatBoost)	Target encoding leaks unless fit inside CV folds — smooth / regularize it
Datetime	Parts (hour, day-of-week, month), cyclical sin/cos, is_weekend / holiday, time-since-event, lags & rolling windows	Lags and rolling stats must use only the past — respect time order
Text	TF-IDF / counts, embeddings, length & keyword flags	High-dim sparse → TruncatedSVD, not PCA; tokenization choices matter
Across features	Interactions (products / ratios), group aggregates (mean-by-category), missingness flags	Aggregates computed before the split leak — compute within train / CV
Missingness as signal	"was_missing" indicator alongside imputing the value	The indicator helps when missingness is informative; still impute

The golden rule of feature engineering. Anything that *learns* from the data — scaling statistics, target encodings, aggregates, imputation values — must be fit on the training fold only, inside cross-validation, never on the full dataset. That one discipline prevents most leakage.

RUBRIC 07

Scoring candidates (don't pick on accuracy alone)

Once you have 2–3 trained candidates, score each across these axes and weight them by what the project actually needs. A model that wins on accuracy but can't be explained may still lose.

CRITERION	WHAT IT TELLS YOU	WHO TENDS TO WIN
Predictive performance	CV score on the *right* metric	`Boosting (tabular)`
Interpretability	Can you explain one prediction?	`Linear · single tree`
Training cost	Time / compute to fit & tune	`Linear · RF (parallel)`
Inference latency	Speed at predict time	`Linear · small tree`
Missing values	Handles NaNs natively?	`LightGBM · XGBoost · HistGB`
Categorical handling	Native, or needs encoding?	`CatBoost`
Outlier robustness	Sensitive to extreme rows?	`Tree ensembles`
Deployment ease	Size, deps, serving complexity	`Linear · RF`
Business explainability	Will stakeholders trust it?	`Linear · trees + SHAP`
Calibration quality	Are predicted probabilities trustworthy? (reliability curve / Brier score)	`Logistic Reg · calibrated ensembles`

CHECKLIST 08

EDA, preprocessing & feature engineering

The unglamorous steps that decide whether the model above is trustworthy.

EDA `explore`

- ✓ **Shape & dtypes** — rows, columns, what each is
- ✓ **Target distribution** — and class imbalance if classifying
- ✓ **Missing-value map** — which columns, how much, why
- ✓ **Numeric distributions** — skew, outliers, scale
- ✓ **Categorical cardinality** — how many levels each
- ✓ **Correlations** — and check for target leakage
- ✓ **Date / time ranges** — gaps, ordering
- ✓ **Duplicates** — exact and near-duplicate rows

Preprocessing `prepare`

- ✓ **Split first** — fit transforms on train only (no leakage)
- ✓ **Missing values** — impute, or use a NaN-aware model
- ✓ **Scaling** — only for distance / linear models; trees don't need it
- ✓ **Encode categoricals** — one-hot, target, or native (CatBoost)
- ✓ **Outliers** — handle only if using a linear model
- ✓ **Feature engineering** — ratios, dates, domain features
- ✓ **Time series** — split by time, build lag / rolling features

RED FLAGS 09

Data leakage — the biggest practical failure mode

Leakage is when information that wouldn't be available at prediction time sneaks into training. The model looks brilliant in validation and then collapses in production. If a result seems too good, suspect leakage first.

Watch for `leakage`

- ⚠ **Future columns** — values recorded after the moment you'd actually predict
- ⚠ **Post-outcome fields** — anything filled in *because* the outcome happened (e.g. tip amount when predicting fare at pickup)
- ⚠ **IDs that encode the target** — case numbers, account types, or codes that quietly leak the label
- ⚠ **Duplicate entities across splits** — the same patient / customer in both train and test inflates the score
- ⚠ **Aggregates computed before splitting** — means, target encodings, or scalers fit on the whole dataset leak test info into train
- ⚠ **Time bleed** — random shuffles on time-ordered data let the model see the future

> The fix is almost always the same: **split first**, then fit every transform (imputation, scaling, encoding, aggregates) on the training fold only — inside cross-validation, not before it.

SOURCE

About the example dataset

The taxi-fare example throughout uses real, public data.

NYC TLC Yellow Taxi Trip Records. Published monthly by the New York City Taxi & Limousine Commission in Parquet format, with fields for pickup/dropoff times and zones, trip distance, passenger count, and itemized fares. (Note: 2025-onward files add a congestion-fee column, so schemas can shift slightly year to year.)

Official source: nyc.gov/site/tlc/about/tlc-trip-record-data.page

---

Routing reference, not a rulebook. Every branch is a strong default to start from — validation results, data quirks, and the cost of errors settle the final choice.